

Particulate Matter & Air Quality Sensor (SPS-30)

Hardware Overview

What is it?

The **SPS-30** is a scientific-grade optical particle counter. It acts as an *air quality monitor*, pulling in environmental air to measure the microscopic specks of dust, smoke, and pollen floating around.

Electrical Explanation

A fan inside the sensor pulls air through a chamber. As the air passes, a laser beam illuminates the particles.

- **Laser Scattering:** When a particle passes through the laser a photodiode catches this light. By measuring the flash, the sensor knows how big the particle was!
- **Categorization:** It sorts these particles based on their size: PM1 . 0 (tiny smoke), PM2 . 5 (fine dust/exhaust), PM4 . 0, and PM10 . 0 (large pollen/heavy dust).

Key Hardware Facts:

- **Connection:** I²C Bus (SDA/SCL).
- **Default Address:** 0x69.
- **Voltage:** Operates at 5V.
- **Signal Flow:** Air → Internal Fan → Laser Beam → Photodiode → I²C Data → Processor.

Software & Programming

kcomp_SPS30.h

Parsing the raw data from the sensor is difficult. This library handles reading, parsing, and math.

Usage: `#include <kcomp_SPS30.h>`

The Data Structure

Unlike other sensors that give you a single number (like `int`), the library gives you a `struct` (a package of variables). When you call `readSPS30()`, you get a package containing all of this:

Field(s)	Type	Description
<code>is_valid</code>	<code>bool</code>	true if data was successfully downloaded.
<code>pm1_0</code> , <code>pm2_5</code> , <code>pm4_0</code> , <code>pm10_0</code>	<code>uint16_t</code>	Mass Concentration: Particle weight in $\mu\text{g}/\text{m}^3$.
<code>nc0_5</code> , <code>nc1_0</code> , <code>nc2_5</code> , <code>nc4_0</code> , <code>nc10_0</code>	<code>uint16_t</code>	Number Concentration: Particle count per cm^3 .
<code>typical_particle_size</code>	<code>float</code>	Average size of the particles in the air in μm .

The Enums (Smart Categories)

Before reading the data, understand the categories (`enum`) the library uses to describe the air:

- **SPS30_AQI_t:** Represents the official *US EPA Air Quality Index*. Possible values are:
`SPS30_AQI_GOOD`, `SPS30_AQI_MODERATE`, `SPS30_AQI_UNHEALTHY_SENSITIVE`,
`SPS30_AQI_UNHEALTHY`, `SPS30_AQI_VERY_UNHEALTHY`, `SPS30_AQI_HAZARDOUS`.
- **SPS30_Source_t:** Guesses the pollution based on particle sizes. Possible values are:
`SPS30_SOURCE_CLEAN`, `SPS30_SOURCE_COMBUSTION`, `SPS30_SOURCE_DUST`, `SPS30_SOURCE_MIXED`.

Function Reference

Function Name	Description & Parameters
<code>bool initSPS30()</code>	Setup: Starts I ² C and spins up the sensor's internal fan. ▪ Returns <code>true</code> if connected, <code>false</code> if missing.
<code>sps30_data_t readSPS30()</code>	Read Data: Asks the sensor for new data and returns the <code>sps30_data_t</code> struct described above.
<code>SPS30_AQI_t getSPS30_AirQualityLevel()</code>	Smart AQI: Returns an enum representing the official <i>US EPA air quality index</i> (e.g., <code>SPS30_AQI_GOOD</code>).
<code>SPS30_Source_t getSPS30_PollutionSource()</code>	Smart Detective: Returns an enum guessing the pollution source (e.g., <code>SPS30_SOURCE_COMBUSTION</code>).
<code>uint8_t getSPS30_AirQualityScore()</code>	Clean Score: Returns a simple 0 (heavily polluted) to 100 (perfectly clean) integer score.

⚠ Pitfalls & Troubleshooting

The "Candle Flatline" (Hardware Limits)

- If you look at the particle data, you will see it shoot up and then freeze at 65535 if the air is very polluted. The sensor isn't broken! E.g. a candle produces a vast amount of particles.
Reason: To save memory, this library runs the SPS-30 in "16-bit Integer Mode." The absolute highest number a 16-bit integer can hold is 65535.

📄 Sample Program

This program initializes the sensor, retrieves data, and prints the air quality to the Serial Monitor.

```
#include <kcomp_SPS30.h>

void setup() {
  Serial.begin(9600); delay(1000);
  Serial.println("K-Comp Smart Air Quality Monitor");
  if (!initSPS30()) { // 1. Initialize the sensor hardware
    Serial.println("Error: SPS30 not found. Check wiring!");
    while(1); // Stop here
  }
}

void loop() {
  // 2. Read Values
  sps30_data_t air = readSPS30();
  // Only process if the data is fresh and valid
  if (air.is_valid) {
    Serial.print("\nPM2.5 Level: "); // 3. Print the Raw PM2.5 Data
    Serial.print(air.pm2_5); Serial.println(" ug/m3");
    Serial.print("Quality Score: "); // 4. Print the "Clean Score" (0-100)
    Serial.print(getSPS30_AirQualityScore()); Serial.println(" / 100");
    // 5. Use the Smart Detective Enum to trigger warnings!
    SPS30_Source_t source = getSPS30_PollutionSource();
    if (source == SPS30_SOURCE_COMBUSTION) {
      Serial.println("Warning: Smoke or Exhaust detected!");
    } // ... more categories available
  }
  delay(500); // Wait half a second before checking again
}
```

💡 **Try it out:** You can find the full sample program `SimpleParticleSensorSPS30` in the K-Comp examples folder!